ECON 3420A Assignment 1: Build Your Own Portfolio

Lecture Note Part 1-3 | 100 marks

Student ID 1155209814

2026-10-16

(a) My beta prediction (for Q2)

Record this **before** you see any regression results.

Prediction	My answer and reason	What I later observed
Which stock will have the highest beta?	MSTR. Its monthly volatility is about 31%, roughly 7 times SPY's 4.5% and far above AMD (20%), NVDA (14.5%), META (12.4%) and LLY (9.1%). Beta scales with a stock's volatility relative to the market's, and MSTR's price is driven by bitcoin and leverage, which should amplify market risk-on and risk-off moves. Second guess: AMD. LLY, a defensive health care stock, should have the lowest beta.	MSTR had the highest beta, $\hat{\beta} = 3.54$ (NVDA 2.21, AMD 2.46, META 1.15, LLY 0.45). Prediction supported.

Prediction recorded on: 2026-10-16 00:28 (based only on the Q1 descriptive statistics; no regression was run)

Setup (supplied)

Configuration

Edit **only** this cell. Tickers must match the CSV column headers exactly, and all assets must use the same currency. For your own downloaded data, see `data_guide.pdf` for the settings to change.

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import portfolio_model as model

config = {
    "stocks": ["LLY", "META", "MSTR", "NVDA", "AMD"],
    "benchmark": "SPY",
    "currency": "USD",
    "prices_file": "data_my_stocks/prices_daily.csv",
    "riskfree_file": "data_my_stocks/riskfree_daily.csv",
    "riskfree_date_kind": "actual quote date",
```

```
    "first_return_month": "2021-10",
    "last_return_month": "2026-09",
    "as_of_date": "2026-10-15",
    "preferences_file": "data/certainty_equivalents.csv",
}
stocks = config["stocks"]
benchmark = config["benchmark"]
asset_columns = stocks + [benchmark]
```

Data files

- `data/prices_daily.csv`: a `date` column (`YYYY-MM-DD`), then the five stocks and the benchmark. Use adjusted prices. Leave missing prices blank; never fill them with zero or the previous value.
- `data/riskfree_daily.csv`: `date,annual_yield_percent`, where 2.7% is stored as `2.7`. Each return month needs a quote dated in the month before it. For your own data, use the quote's actual date and set `riskfree_date_kind` to `actual quote date`.
- The supplied Hong Kong files are **practice data**. Their bill dates are period labels, not verified quote dates; say so if you use them.

The cell below aligns the data for you. It uses the last day each month with prices for all six assets, requires at least 60 complete monthly returns ending no later than September 2026, and stops with an error message if a month or bill quote is missing.

```
Common sample: 2021-10 to 2026-09; T=60 monthly returns.
Currency declared by student: USD.
Latest bill-series date: 2026-09-30; annual decimal=0.042000.
Bill date meaning: actual quote date.
R contains raw simple returns; X contains excess returns. Do not subtract cash twice.
```

`R` holds monthly returns and `X` holds monthly **excess** returns. The code checks dates and arithmetic, but you must check that the prices are adjusted and that the bill matches your currency.

1. Choose stocks and calculate returns (15 marks)

Task to Student:

Name	Ticker	Sector	Reason
Eli Lilly	LLY	Health care (pharmaceuticals)	Defensive growth from obesity and diabetes drugs; low link to tech.
Meta Platforms	META	Communication services	Large-cap digital advertising and AI spending.
Strategy (MicroStrategy)	MSTR	Information technology (software / bitcoin treasury)	Very high-volatility, bitcoin-linked stock.
NVIDIA	NVDA	Information technology (semiconductors)	Leader in AI chips; high growth and risk.
Advanced Micro Devices	AMD	Information technology (semiconductors)	Chip rival to NVIDIA; shows how same-sector stocks co-move.

Name	Ticker	Sector	Reason
Benchmark: SPDR S&P 500 ETF	SPY	Broad US market	Tracks the S&P 500; a comparison, not a holding.

All six assets are US-listed and priced in USD.

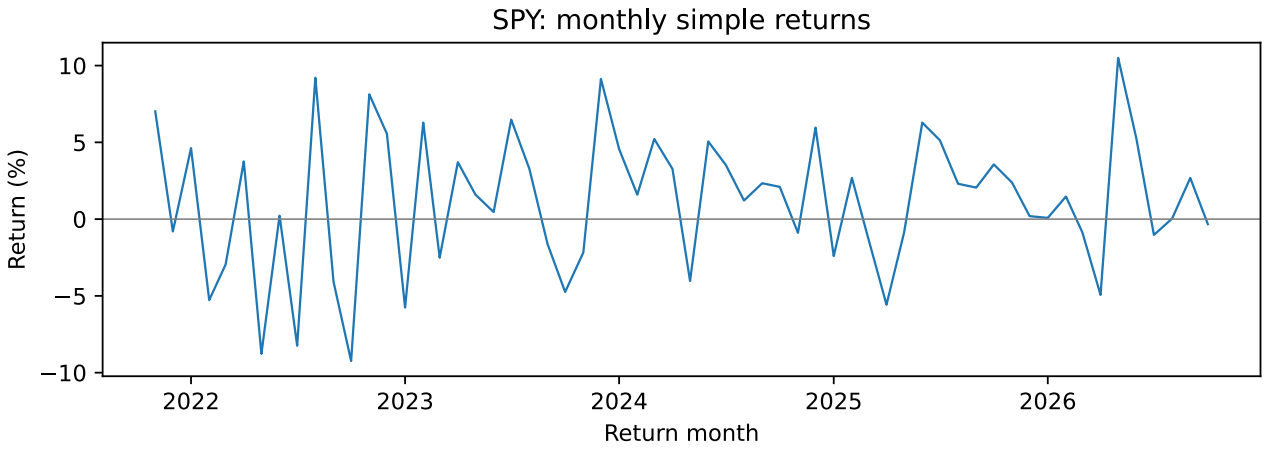
Task to AI: show each asset's average monthly return, volatility and average excess return. The example below is supplied; `show_table` prints a table in both HTML and PDF, and you can reuse it later.

Asset	Mean return (%)	Volatility (%)	Mean excess return (%)
LLY	3.193	9.094	2.871
META	2.061	12.350	1.739
MSTR	5.825	31.137	5.503
NVDA	5.092	14.459	4.770
AMD	4.739	20.033	4.416
SPY	1.176	4.523	0.854

The numbers match the stock descriptions above. The three AI and tech names and bitcoin-linked MSTR had both the highest average returns (MSTR 5.8%, NVDA 5.1%, AMD 4.7% per month) and the highest volatility (MSTR 31%, AMD 20%, NVDA 14.5%). Defensive LLY earned a strong 3.2% a month with the lowest stock volatility (9.1%). Every stock was far more volatile than SPY (4.5%), the diversified market benchmark, which returned 1.2% a month.

The chart below is a plotting example only; it is not marked.

```
 #| label: fig-benchmark-returns
#| fig-cap: "Benchmark monthly simple returns in the aligned sample."
#| fig-width: 8
#| fig-height: 3
fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(R.index.to_timestamp(how="end"), 100 * R[benchmark], linewidth=1)
ax.axhline(0, color="grey", linewidth=0.7)
ax.set(title=f"{benchmark}: monthly simple returns",
       xlabel="Return month", ylabel="Return (%)")
fig.tight_layout()
plt.show()
```



2. Estimate and understand alpha and beta (20 marks)

$$X_{i,t} = \alpha_i + \beta_i X_{M,t} + e_{i,t}, \qquad X_{i,t} = R_{i,t} - R_{f,t}, \quad X_{M,t} = R_{M,t} - R_{f,t}.$$

Task to AI: complete `estimate_index_model`, then switch on the cell below. Add one chosen stock's scatter plot with its fitted SCL, with $X_{M,t}$ on the horizontal axis and $X_{i,t}$ on the vertical axis.

```
estimates, residuals = model.estimate_index_model(X, stocks, benchmark)
show_table(estimates.rename(columns={
    "alpha": r"$\hat\alpha_i$", "beta": r"$\hat\beta_i$",
    "r_squared": r"$R^2$",
}), decimals=4)
```

	$\hat{\alpha}_i$	$\hat{\beta}_i$	R^2
LLY	0.0249	0.4489	0.0493
META	0.0075	1.1545	0.1783
MSTR	0.0248	3.5433	0.2624
NVDA	0.0288	2.2092	0.4744
AMD	0.0231	2.4643	0.3063

```
chosen_stock = "MSTR"
est = estimates.loc[chosen_stock]
x_pct = 100 * X[benchmark]
y_pct = 100 * X[chosen_stock]
grid = np.linspace(x_pct.min(), x_pct.max(), 100)
fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(x_pct, y_pct, s=18, alpha=0.7)
ax.plot(grid, 100 * est["alpha"] + est["beta"] * grid, color="red",
        label=f"Fitted SCL: beta = {est['beta']:.2f}, $R^2$ = {est['r_squared']:.2f}")
ax.set(title=f"{chosen_stock} vs {benchmark}: security characteristic line",
       xlabel=f"{benchmark} monthly excess return (%)",
       ylabel=f"{chosen_stock} monthly excess return (%)")
ax.legend()
fig.tight_layout()
plt.show()
```

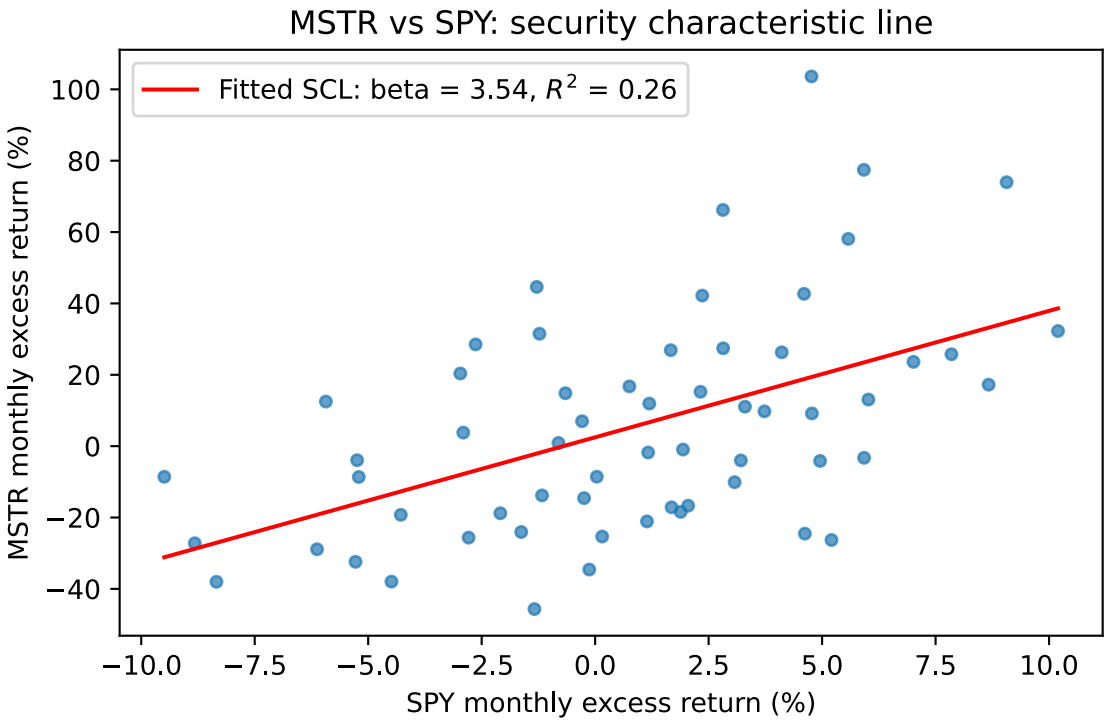


Figure 1: MSTR monthly excess returns against SPY monthly excess returns, with the fitted security characteristic line.

Task to Student:

I chose **MSTR** (60 months, October 2021 to September 2026).

- **Alpha:** $\hat{\alpha} = 0.0248$, i.e. about 2.5% per month of excess return that the market does not explain. It is imprecise: its standard error is about 3.5% per month ($t \approx 0.70$), so it is not statistically different from zero. I do not read it as reliable skill or mispricing.
- **Beta:** $\hat{\beta} = 3.54$ (standard error 0.78, $t \approx 4.5$). If the market's monthly excess return rises by one percentage point, the line predicts MSTR's excess return rises by about **3.54 percentage points**. A rough 95% interval is 2.0 to 5.1, so MSTR is clearly far more market-sensitive than the market itself ($\beta = 1$).
- **R^2:** 0.26, so the market explains only about a quarter of MSTR's monthly variation. The other 74% is firm-specific, mainly bitcoin price moves and the company's debt and share issuance to buy bitcoin.
- **Why the line misses most months:** the residual $e_{i,t}$ collects news unrelated to the S&P 500. Its standard deviation is about 27% per month, so single months can be far from the line. Beta is also not stable: it is 3.88 in the first 30 months and 2.88 in the last 30, and a straight line cannot capture this or the fat-tailed jumps visible in the scatter plot.
- **My prediction:** supported. MSTR has the highest beta (3.54), AMD is second (2.46) as I guessed, and LLY is lowest (0.45). The standard error on MSTR's beta (0.78) is large, so the gap to AMD is less certain than the point estimates suggest.

3. Understand variance and covariance (20 marks)

$$\sigma_i^2 = \beta_i^2 \sigma_M^2 + \sigma^2(e_i), \qquad \mathbf{Cov}(X_i, X_j) = \beta_i \beta_j \sigma_M^2 \ (i \neq j), \qquad \hat{\sigma}^2(e_i) = \frac{\sum_{t=1}^{T} \hat{e}_{i,t}^2}{T-2}.$$

Task to AI: complete `single_index_covariance`, then switch on the cell below. It shows one 5×5 table of monthly variances (diagonal) and covariances, labelled by ticker.

```
market_variance_m = X[benchmark].var(ddof=1)
covariance_m = model.single_index_covariance(
```

```
    estimates["beta"], X[benchmark], residuals
)
show_table(covariance_m, decimals=6)
```

	LLY	META	MSTR	NVDA	AMD
LLY	0.008403	0.001048	0.003217	0.002006	0.002238
META	0.001048	0.015332	0.008275	0.005159	0.005755
MSTR	0.003217	0.008275	0.097994	0.015834	0.017662
NVDA	0.002006	0.005159	0.015834	0.020999	0.011012
AMD	0.002238	0.005755	0.017662	0.011012	0.040578

Task to Student:

- **One entry and its sign:** $\text{Cov(LLY, MSTR)} = \hat{\beta}_{LLY}\hat{\beta}_{MSTR}\hat{\sigma}^2_M = 0.449 \times 3.543 \times 0.00202 \approx 0.0032$, which is positive. In the single-index model, two stocks are linked only through the market, so the covariance has the sign of $\beta_i\beta_j$. Both betas are positive, so the model says both tend to rise when the market rises. The raw sample covariance is actually negative (about -0.0029). LLY behaved as a defensive stock in months when bitcoin-linked MSTR fell, which the model cannot represent.
- **Why combining stocks reduces risk:** portfolio variance is $\sum_i w_i^2\sigma_i^2 + \sum_{i\neq j} w_i w_j \text{Cov}(X_i, X_j)$. When covariances are small relative to variances, the firm-specific parts partly cancel. The five stocks' annual volatilities average about 61%, but the equally weighted portfolio's is 39.7%. The optimized portfolio's 28.9% is below even the least volatile single stock (LLY, about 32%).
- **Why the zero-residual-covariance assumption can fail:** NVDA and AMD are both AI and semiconductor chip makers. They share industry shocks such as chip export rules, data-centre spending and AI sentiment, which the S&P 500 does not fully capture. Their residual correlation is +0.35 (META and NVDA +0.32), so the model understates their covariance: 0.0110 against a sample value of 0.0171 per month. This understates the risk of holding both.

4. Find and assess a risky portfolio (20 marks)

$$\text{SR}_P = \frac{\mathbb{E}(R_P) - R_f}{\sigma_P}.$$

Task to AI: complete `optimize_risky_portfolio`, then switch on the cell below. Annualize by multiplying monthly mean excess returns and every covariance entry by 12. Show the five weights (nonnegative, summing to 100%), annual excess return, volatility and Sharpe ratio next to the equally weighted portfolio.

This cell also saves your portfolio to `outputs/q5_portfolio.json` for Question 5.

```
annual_excess_means = 12 * X[stocks].mean()
annual_covariance = 12 * covariance_m
portfolio = model.optimize_risky_portfolio(annual_excess_means, annual_covariance)
equal_w = pd.Series(1 / len(stocks), index=stocks)
opt_w = portfolio["weights"].reindex(stocks)
eq_excess = float(equal_w @ annual_excess_means)
eq_vol = float(np.sqrt(equal_w @ annual_covariance.loc[stocks, stocks] @ equal_w))
eq_sharpe = eq_excess / eq_vol
assert (opt_w >= -1e-9).all() and abs(opt_w.sum() - 1) < 1e-9
assert portfolio["sharpe"] >= eq_sharpe - 1e-9
weights_table = pd.DataFrame({
```

```
    "Optimized weight (%)": 100 * opt_w,
    "Equal weight (%)": 100 * equal_w,
})
weights_table.index.name = "Stock"
show_table(weights_table, decimals=2)
stats_table = pd.DataFrame({
    "Annual excess return (%)": [100 * portfolio["excess_return"], 100 * eq_excess],
    "Annual volatility (%)": [100 * portfolio["volatility"], 100 * eq_vol],
    "Sharpe ratio": [portfolio["sharpe"], eq_sharpe],
}, index=["Optimized", "Equal weight"])
stats_table.index.name = "Portfolio"
show_table(stats_table, decimals=3)

# Save the exact portfolio and sample used to generate Question 5's menu.
import hashlib
import json
from risk_aversion_quiz import normalize_portfolio_inputs
q5_inputs = normalize_portfolio_inputs({
    "schema_version": 2,
    "weights": portfolio["weights"].to_dict(),
    "annual_premium": portfolio["excess_return"],
    "annual_volatility": portfolio["volatility"],
    "annual_safe_rate": rf_current,
    "currency": config["currency"],
    "safe_rate_date": str(latest_quote_date.date()),
    "safe_rate_date_kind": config["riskfree_date_kind"],
    "first_return_month": str(X.index[0]),
    "last_return_month": str(X.index[-1]),
    "n_months": len(X),
    # Fixed line ending so Windows and Mac give the same fingerprint.
"data_fingerprint": hashlib.sha256(X.to_csv(lineterminator="\n").encode()).hexdigest(),
})
Path("outputs").mkdir(exist_ok=True)
Path("outputs/q5_portfolio.json").write_text(
    json.dumps(q5_inputs, indent=2), encoding="utf-8"
)
print("Saved outputs/q5_portfolio.json for Question 5.")
```

Stock	Optimized weight (%)	Equal weight (%)
LLY	54.82	20.00
META	3.22	20.00
MSTR	2.16	20.00
NVDA	31.95	20.00
AMD	7.86	20.00

Portfolio	Annual excess return (%)	Annual volatility (%)	Sharpe ratio
Optimized	43.434	28.914	1.502

Portfolio	Annual excess return (%)	Annual volatility (%)	Sharpe ratio
Equal weight	46.317	39.662	1.168

```
Saved outputs/q5_portfolio.json for Question 5.
```

Task to Student:

- **Checks:** all five weights are nonnegative and sum to 100%. The optimized Sharpe ratio is 1.502, above the equally weighted 1.168 (and SPY's 0.66 over the same period), as it must be since equal weighting is one of the portfolios the optimizer could have chosen.
- **Holdings:** the portfolio is concentrated: LLY 54.8% and NVDA 32.0% make up 87%. MSTR gets only 2.2% even though it had the highest average excess return (66% a year), because its 108% annual volatility makes it a poor trade-off. LLY gets the most because it combines a high historical mean (34% a year) with a low beta, so it diversifies the three tech stocks.
- **Why history cannot guarantee the future:**
  - Five years gives very imprecise means. The standard error of an annual mean is roughly volatility$/\sqrt{5}$: about 14% for LLY and 48% for MSTR. The optimizer treats these noisy means as exact and puts large weights on whichever stock looked best.
  - The inputs are unstable. LLY's annual excess return was 51% in the first half of the sample and 18% in the second, and AMD's beta rose from 2.0 to 3.5.
  - I chose these stocks knowing they had done well, which biases their historical returns upward. A Sharpe ratio of 1.5 would be exceptional for an investor choosing at the start of the period.
  - The single-index model misses industry links (see Question 3), and the period includes regime changes such as the 2022 interest-rate rise and the AI boom.

Now render the report once. This creates `outputs/q5_portfolio.json`. If $\delta \leq 0$, keep it and explain it in Question 6; do not replace it with a made-up positive return.

5. Work out your risk aversion (10 marks)

P is your Question 4 portfolio, with annual excess return δ, variance σ_P^2 and mean $\mu_P = R_f + \delta$. For $\delta > 0$, let $G = \delta/\sigma_P^2$. The quiz offers five risk-aversion levels $\gamma_m = mG$ (normally $m = 1.25, 2, 3, 4, 5$) and shows each as a certainty equivalent wealth:

$$c_m = \mu_P - \frac{\gamma_m \sigma_P^2}{2}, \qquad W_m = 10000(1 + c_m).$$

Task to AI: run the quiz in a terminal (not inside this file):

```
python risk_aversion_quiz.py --portfolio outputs/q5_portfolio.json
```

It shows P's mean and volatility, its lowest 10%, middle 80% and highest 10% return/wealth ranges, and the menu of W_m labelled A to E. No option is below the lowest-10% outcome, so for a high-Sharpe portfolio the quiz uses a narrower range of m. After the student types a letter, it prints the implied γ_m and saves the choice to `data/certainty_equivalents.csv`. Then complete `infer_risk_aversion` and switch on the cell below, which reads that file.

```
from risk_aversion_quiz import load_preferences
preference_trials = None
gamma = None
if portfolio["excess_return"] > 0:
```

```
    preference_trials = load_preferences(config["preferences_file"], q5_inputs)
    preferences = model.infer_risk_aversion(preference_trials)
    gamma = preferences["gamma"]
    preference_estimates = preferences["estimates"]
    preference_table = pd.DataFrame({
        "Choice": preference_estimates["choice"],
        "CE wealth": preference_estimates["certainty_equivalent_wealth"],
        "CE (%)": 100 * preference_estimates["certainty_equivalent"],
        "Gamma": preference_estimates["gamma"],
    }).set_index("Choice")
    show_table(preference_table, decimals=4)
    print(f"Implied gamma for my selected portfolio: {gamma:.6f}")
else:
    print("Nonpositive expected excess return: numeric gamma is not elicited.")
```

Choice	CE wealth	CE (%)	Gamma
A	12284.9389	22.8494	5.9291

```
Implied gamma for my selected portfolio: 5.929109
```

If you change your stocks, data or Question 4 portfolio, run the quiz again: the cell rejects a choice made for a different portfolio.

Task to Student:

- **The menu:** the quiz used P's mean 47.63% and volatility 28.91%, with the safe rate of 4.20% dated 30 September 2026. For 10,000 USD, the lowest 10% of outcomes is below 11,058 USD (+10.6%) and the highest 10% above 18,469 USD (+84.7%).
- **My choice:** I chose **A (12,284.94 USD)**, which implies $\gamma = 5.93$. That is 1.14 times $G = \delta/\sigma_P^2 = 5.20$. I have a long investment horizon and no near-term need for the money, and even the lowest-10% outcome is a gain under this approximation. So I would give up a sure 12,285 USD only for a portfolio worth noticeably more.
- **What a larger gamma means:** stronger dislike of variance. The penalty $\gamma\sigma_P^2/2$ grows, the certainty equivalent falls, and the investor holds less of P.
- **How well the menu fits:** it is a coarse estimate. Because P's historical Sharpe ratio is very high, the quiz capped the choices: the multipliers only range from 1.14 to 1.71, so every option implies a risky share between 59% and 88%. A is the edge of the menu, so my true γ may be even lower than 5.93. The bell-curve ranges also understate the crash risk of MSTR, NVDA and AMD.

6. Choose your stock-and-safe-asset mix (15 marks)

The **final portfolio** F puts a share w_r^* of wealth in P (keeping its Question 4 weights w_i) and $1 - w_r^*$ in the safe asset, using your Question 5 γ:

$$w_r^* = \frac{\delta}{\gamma\sigma_P^2}, \qquad \mathbb{E}(R_F) = R_f + w_r^*\delta, \qquad \sigma_F = w_r^*\sigma_P.$$

Task to AI: complete `allocate_portfolio`, then switch on the cell below. For wealth of 10,000, show w_r^*, each stock's percentage $w_r^* w_i$ and amount $10000\, w_r^* w_i$, the safe asset's percentage $1 - w_r^*$ and amount, and $\mathbb{E}(R_F)$ and σ_F. Do not clip w_r^*.

```
allocation_premium = portfolio["excess_return"]
allocation_volatility = portfolio["volatility"]
mix = model.allocate_portfolio(
    portfolio["weights"], allocation_premium, allocation_volatility,
    gamma, rf_current, 10000
)
if allocation_premium > 0:
    assert 0 < mix["risky_share"] < 1
else:
    assert mix["risky_share"] == 0
holdings_table = pd.DataFrame({
    "Weight (%)": 100 * mix["holdings"]["weight"],
    "Amount": mix["holdings"]["amount"],
})
holdings_table.index.name = "Holding"
show_table(holdings_table, decimals=2)
mix_table = pd.DataFrame({
    "Value": [100 * mix["risky_share"], 100 * mix["expected_return"],
              100 * mix["volatility"]],
}, index=["Risky share w* (%)", "Expected annual return E(R_F) (%)",
          "Annual volatility sigma_F (%)"])
mix_table.index.name = "Final portfolio F"
show_table(mix_table, decimals=2)
```

Holding	Weight (%)	Amount
LLY	48.03	4803.39
META	2.82	281.76
MSTR	1.89	189.21
NVDA	27.99	2799.22
AMD	6.89	688.70
Safe asset	12.38	1237.71

Final portfolio F	Value
Risky share w* (%)	87.62
Expected annual return E(R_F) (%)	42.26
Annual volatility sigma_F (%)	25.34

Task to Student (prediction, recorded before the next cell was switched on): w_r^* should **halve exactly**, from 87.6% to about 43.8%. The formula $w_r^* = \delta/(\gamma\sigma_P^2)$ is proportional to δ when γ and σ_P^2 are fixed: the reward per unit of risk halves while the penalty for risk stays the same. The stock weights inside P do not change, so every stock's dollar holding halves too. The expected return of F should fall by three quarters of its premium, because $w_r^*\delta$ falls to one quarter: from $4.2\% + 38.1\%$ to about $4.2\% + 9.5\% = 13.7\%$.

Task to AI: switch on the cell below and compare the original and new w_r^*.

```
half_mix = model.allocate_portfolio(
    portfolio["weights"], 0.5 * allocation_premium,
```

```
    allocation_volatility, gamma, rf_current, 10000
)
comparison = pd.DataFrame({
    "Original": [100 * mix["risky_share"], 100 * mix["expected_return"],
                 100 * mix["volatility"]],
    "Half premium": [100 * half_mix["risky_share"], 100 * half_mix["expected_return"],
                     100 * half_mix["volatility"]],
}, index=["Risky share w* (%)", "Expected annual return (%)",
          "Annual volatility (%)"])
comparison.index.name = "Result"
show_table(comparison, decimals=2)
```

Result	Original	Half premium
Risky share w* (%)	87.62	43.81
Expected annual return (%)	42.26	13.71
Annual volatility (%)	25.34	12.67

Nonpositive-premium exception: if $\delta \leq 0$, report 100% in the safe asset, expected return R_f and zero volatility; the half-premium test also gives 0%. Explain why positive risk aversion cannot give an interior allocation.

Task to Student:

- **Risk aversion and the risky share:** $w_r^* = \delta/(\gamma\sigma_P^2)$ is inversely proportional to γ. My $\gamma = 5.93$ gives $w_r^* = 87.6\%$, so 8,762 USD goes in P and 1,238 USD in the safe asset. Doubling γ would halve the share. The most risk-averse option on the menu, E, would have given 58.6%.
- **Prediction against result:** my prediction was confirmed exactly. The risky share fell from 87.6% to 43.8%, every stock's holding halved, and the safe holding rose to 5,619 USD. $\mathbb{E}(R_F)$ fell from 42.3% to 13.7%, and σ_F from 25.3% to 12.7%.
- **Caveats:**
  - Because the menu sets $\gamma = mG$ with $G = \delta/\sigma_P^2$, my share is simply $1/m = 1/1.14$. The quiz choice, not the data, fixes the 87.6%.
  - μ_P adds today's 4.2% bill rate to a premium measured when the bill rate averaged about 3.9%.
  - The menu's return ranges assume normal returns, which understates the crash risk of MSTR, NVDA and AMD.

Would I accept it? (at most 100 words) No, not as estimated. The 87.6% risky share rests on a 43% expected excess return and a 1.5 Sharpe ratio. Those figures come from five unusually strong years for stocks I chose knowing they had performed well. Their standard errors are huge, and the half-premium test shows the share moves one-for-one with that premium. I would shrink expected returns toward a market-like premium of about 5–8%. That would cut the risky share to well below 50%. I would also cap LLY near 25% and use a covariance model that captures the chip-industry link.

(b) AI evidence entry (for Q6)

Use the half-premium test from Question 6.

Item	My entry
AI tool and relevant claim	GitHub Copilot (Claude): "$w_r^* = \delta/(\gamma\sigma_P^2)$ is proportional to δ, so halving δ with γ and σ_P^2 fixed halves the risky share exactly, from 87.6% to about 43.8%."

Item	My entry
My prediction before testing	w_r^* halves exactly (87.6% to about 43.8%), because it is proportional to δ when γ and σ_P^2 are fixed. $\mathbb{E}(R_F)$ falls to about 13.7%.
My test and result	Fixed: stock weights (LLY 54.8%, META 3.2%, MSTR 2.2%, NVDA 32.0%, AMD 7.9%), $\sigma_P = 28.91\%$, $\gamma = 5.93$, $R_f = 4.20\%$, wealth 10,000 USD. δ changed from 43.43% to 21.72%. w_r^* fell from 87.62% to 43.81%, $\mathbb{E}(R_F)$ from 42.26% to 13.71%, and σ_F from 25.34% to 12.67%.
My decision	I accept the AI's claim about the formula: the test reproduced it exactly. I revise the recommendation itself. The allocation is only as good as δ, and δ is a noisy, upward-biased historical estimate. So I would shrink δ toward a market premium, which gives a much smaller risky share, and limit concentration in LLY.

(c) Data sources (for Q1)

- **Prices:** Yahoo Finance via `yfinance` (`tools/download_data.py`), daily prices adjusted for splits and dividends.
- **Assets:** LLY, META, MSTR, NVDA, AMD; benchmark SPY; all in USD.
- **Sample:** 60 monthly returns, October 2021 to September 2026, from the last common trading day of each month (the starting price is from September 2021).
- **Safe rate:** FRED 3-month Treasury yield (DGS3MO), using each month's return with the previous month's last quote, dated by actual quote date. Monthly rate is the annual percent divided by 1200.
- I did **not** use the supplied HK practice data; the data is in `data_my_stocks/`.

Appendix: evidence for statistics quoted in the text

These tables compute the t-statistics, residual correlations, sample covariances and split-sample estimates quoted in Questions 2–4 from the same X and regression results.

```
T_obs = len(X)
xm = X[benchmark]
sxx = ((xm - xm.mean()) ** 2).sum()
resid_var = (residuals ** 2).sum() / (T_obs - 2)
se_beta = np.sqrt(resid_var / sxx)
se_alpha = np.sqrt(resid_var * (1 / T_obs + xm.mean() ** 2 / sxx))
inference = pd.DataFrame({
    "alpha": estimates["alpha"], "SE alpha": se_alpha,
    "t alpha": estimates["alpha"] / se_alpha,
    "beta": estimates["beta"], "SE beta": se_beta,
    "t beta": estimates["beta"] / se_beta,
    "Residual SD": np.sqrt(resid_var),
})
inference.index.name = "Stock"
show_table(inference, decimals=4)
```

Stock	alpha	SE alpha	t alpha	beta	SE beta	t beta	Residual SD
LLY	0.0249	0.0118	2.1168	0.4489	0.2588	1.7343	0.0894
META	0.0075	0.0148	0.5099	1.1545	0.3254	3.5482	0.1124
MSTR	0.0248	0.0354	0.6999	3.5433	0.7799	4.5429	0.2694
NVDA	0.0288	0.0139	2.0803	2.2092	0.3053	7.2350	0.1055
AMD	0.0231	0.0221	1.0460	2.4643	0.4869	5.0610	0.1682

```
resid_corr = residuals[stocks].corr()
resid_corr.index.name = "Residual corr."
show_table(resid_corr, decimals=2)
pairs = [("LLY", "MSTR"), ("NVDA", "AMD"), ("META", "NVDA")]
pairs = [(a, b) for a, b in pairs if a in stocks and b in stocks]
sample_cov = X[stocks].cov()
cov_check = pd.DataFrame({
    "Model cov": [covariance_m.loc[a, b] for a, b in pairs],
    "Sample cov": [sample_cov.loc[a, b] for a, b in pairs],
}, index=[f"{a}-{b}" for a, b in pairs])
cov_check.index.name = "Pair (monthly)"
show_table(cov_check, decimals=5)
```

Residual corr.	LLY	META	MSTR	NVDA	AMD
LLY	1.00	−0.13	−0.26	−0.08	−0.05
META	−0.13	1.00	0.06	0.32	0.15
MSTR	−0.26	0.06	1.00	0.22	−0.10
NVDA	−0.08	0.32	0.22	1.00	0.35
AMD	−0.05	0.15	−0.10	0.35	1.00

Pair (monthly)	Model cov	Sample cov
LLY-MSTR	0.00322	−0.00292
NVDA-AMD	0.01101	0.01712
META-NVDA	0.00516	0.00887

```
half = T_obs // 2
first_est, _ = model.estimate_index_model(X.iloc[:half], stocks, benchmark)
second_est, _ = model.estimate_index_model(X.iloc[half:], stocks, benchmark)
annual_vol = np.sqrt(12) * X[stocks].std(ddof=1)
stability = pd.DataFrame({
    "Beta, 1st half": first_est["beta"],
    "Beta, 2nd half": second_est["beta"],
    "Ann. excess, 1st half (%)": 100 * 12 * X[stocks].iloc[:half].mean(),
    "Ann. excess, 2nd half (%)": 100 * 12 * X[stocks].iloc[half:].mean(),
    "SE of ann. mean (%)": 100 * annual_vol / np.sqrt(T_obs / 12),
})
stability.index.name = "Stock"
```

```
show_table(stability, decimals=2)
spy_sharpe = 12 * xm.mean() / (np.sqrt(12) * xm.std(ddof=1))
print(f"{benchmark} annual Sharpe ratio over the sample: {spy_sharpe:.3f}")
print(f"Average monthly bill rate, annualized: {100 * 12 * (R[benchmark] -
X[benchmark]).mean():.2f}%")
```

Stock	Beta, 1st half	Beta, 2nd half	Ann. excess, 1st half (%)	Ann. excess, 2nd half (%)	SE of ann. mean (%)
LLY	0.48	0.43	51.07	17.83	14.09
META	1.00	1.54	24.11	17.62	19.05
MSTR	3.88	2.88	101.29	30.78	48.19
NVDA	2.40	1.82	75.84	38.64	22.35
AMD	1.99	3.54	37.17	68.82	31.02

```
SPY annual Sharpe ratio over the sample: 0.657
Average monthly bill rate, annualized: 3.87%
```